CSE 2218: Data Structure and Algorithms II Laboratory

Sherajul Arifin
Lecturer, Department of Computer Science and Engineering,
United International **University**

Recursion

www.penjee.com

Definition

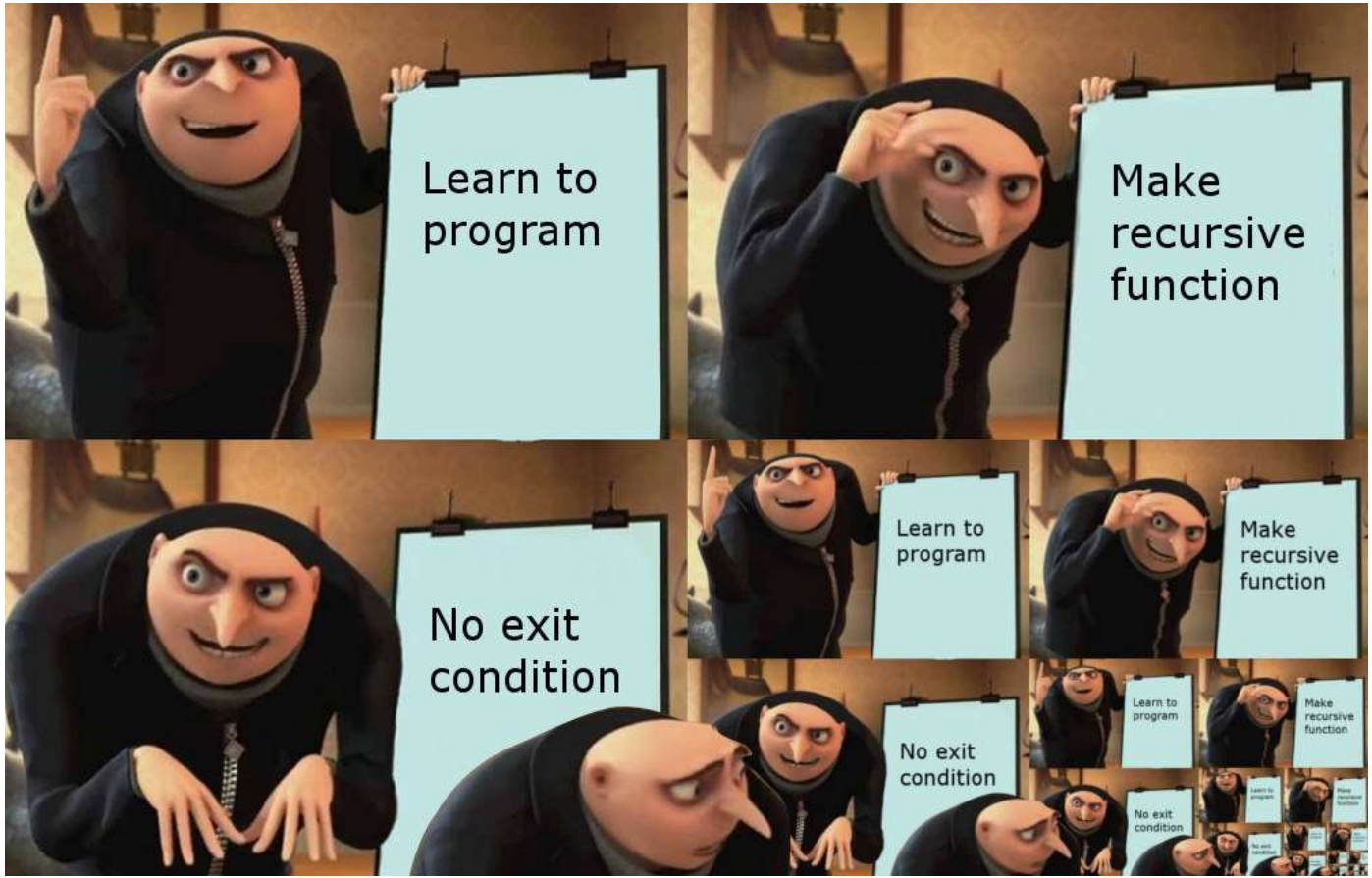
A function that calls itself to solve a problem

Two main components:

- **Base** Case: The stopping condition.
- **Recursive** Case: The function calls itself with modified parameters.

Why Use Recursion?

- Simplifies complex problems.
- Natural fit for problems like divide and conquer.
- Reduces the need for explicit loops and complex iterations.

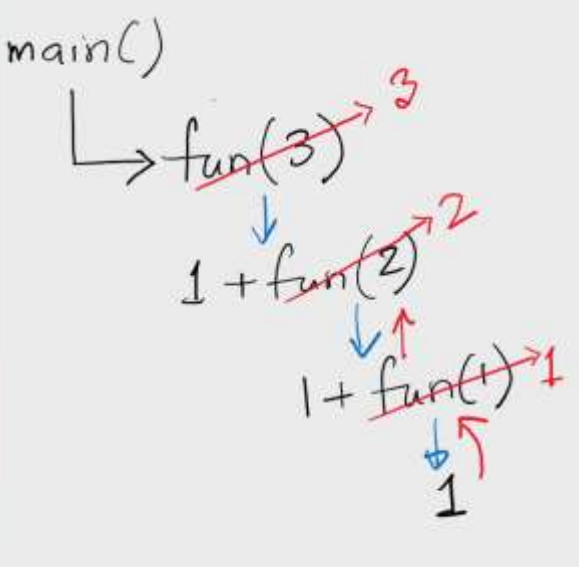

Learn to program
Make recursive function
No exit condition
Learn to program
Make recursive function
No exit condition
Learn to program
Make recursive function
No exit condition

জানতে হবে
কোথায়
থামতে হবে

Simulation of a recursive function:

```
int fun(int n)
{
    if(n == 1)
        return 1;
    else
        return 1 + fun(n-1)
}

int main() {
    int n = 3;
    printf("%d", fun(n));
    return 0;
}
```



How to write recursive functions

1. Divide the problem into smaller subproblems. Identify the recursive formula.

```
   Fact (1) = 1
Fact (2) = 2. 1 = 2. Fact (1)
Fact (3) = 3. 2. 1 = 3. Fact (2)
Fact (4) = 4. 3. 2. 1 = 4. Fact (3)
…
Fact (n) = n. Fact(n-1)
```

How to write recursive functions

2. Specify the base condition to stop the recursion

The base case returns a value without making any subsequent recursive calls. For example: Fact (1) = 1

```
Function rec:
  if base condition:
  ...
  else
  ... recursive formula
```

```
Function Fact(n):
  if n==1 then
    return 1
  else
    return n*Fact(n-1)
```

Factorial Calculation

Write a recursive implementation of the factorial function.

Recall that $n! = 1 \times 2 \times \ldots \times n$, with the special case that $0! = 1$

Factorial Calculation

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0)  // Base case
        return 1;
    return n * factorial(n - 1);  // Recursive case
}

int main() {
    cout << factorial(5);
    return 0;
}
```

Fibonacci

Write a recursive program to print the nth Fibonacci number.

Fibonacci

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n <= 1)  // Base case
        return n;
    return fibonacci(n-1) + fibonacci(n-2);  // Recursive case
}

int main() {
    cout << fibonacci(6);
    return 0;
}
```

Sum of Digits

You are given an integer number. You need to calculate the sum of its digits using recursion.

Sum of Digits

```
#include <iostream>
using namespace std;

int sumDigits(int n) {
    if (n == 0)  // Base case
        return 0;
    return n % 10 + sumDigits(n / 10);  // Recursive case
}

int main() {
    cout << sumDigits(1234);
    return 0;
}
```

Reverse a string

Write a recursive function to reverse a string.

Reverse a string

```
#include <iostream>
using namespace std;

string reverseString(const string &str, int index) {
    if (index < 0)
        return "";
    return str[index] + reverseString(str, index - 1);
}

int main() {
    string str = "hello";
    string reversed = reverseString(str, str.length() - 1);
    cout << "Reversed string: " << reversed << endl;

    return 0;
}
```

Task

1. Write a recursive program to count the number of digits of an integer.
2. Write a recursive program to calculate the power of x (x^y), where y is a non-negative integer.
3. Write a recursive program to check if a given string is a palindrome or not (not case sensitive, ignore whitespaces)

Task

4. Write a recursive implementation of binary search in a sorted array.

5. Write a recursive program to find the GCD of x and y where x, y are positive integers. (Hint: use Euclid's algorithm)

6. Write a recursive program to print an array of size n in reverse order.

Task

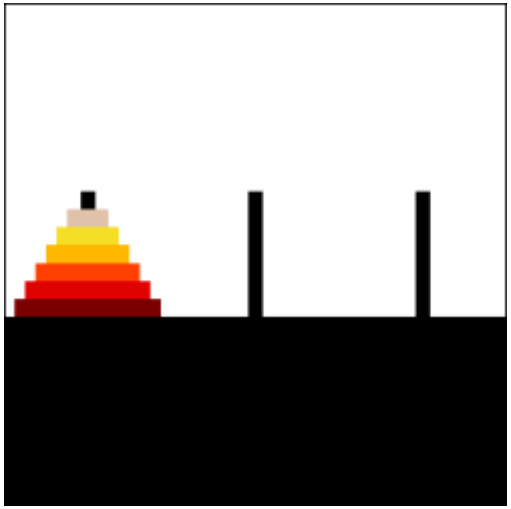
7. Find the sum of the following series up to n^{th} position: $1^2+2^2+3^2+\ldots$

8. Write a recursive program to check if a given positive integer is a palindrome or not. An integer is a palindrome when it reads the same backward as forward.

9. Write a recursive program to print all subsets of a set of n elements.

Task

10. Write a recursive program to solve the Tower of Hanoi problem for n disks.



Sample input	Sample output
4	Move disk 1 from A to B Move disk 2 from A to C Move disk 1 from B to C Move disk 3 from A to B Move disk 1 from C to A Move disk 2 from C to B Move disk 1 from A to B Move disk 4 from A to C Move disk 1 from B to C Move disk 2 from B to A Move disk 1 from C to A Move disk 3 from B to C Move disk 1 from A to B Move disk 2 from A to C Move disk 1 from B to C

Thank you!